Trabalho 3 - Remoção em Tabela Hash com Rehashing

Disciplina: Resolução de Problemas Estruturados em Computação **Turma:** BES 4ºp manhã

- Mariana Schneider Sobrinho - 40107348
- Heitor Roberto Gonçalves - 40113496
- Josue A. Nonalaya - 40109965

1. Implementação

funcaoHash: retorna (chave % tamanho)

rehash retorna (posicao + 1) % tamanho.

criaTabela cria new int[tamanho] e preenche todas as posições com VAZIO usando for.

busca parte de funcaoHash, incrementa acessos a cada posição examinada e para ao encontrar a chave ou uma posição VAZIO; o laço é limitado por passos < tamanho, o que evita laço infinito.

insere: percorre o caminho até um VAZIO ou até completar uma volta, guarda a primeira posição REMOVIDO e só grava depois de confirmar que a chave não existe. Retorna a posição, -1 (duplicada) ou -2 (cheia), incrementando reaproveitamentos quando grava sobre REMOVIDO.

removeIngenua: chama busca e grava VAZIO;

remove: chama busca e grava REMOVIDO.

reorganiza: cria uma tabela nova, reinsere os valores maiores ou iguais a zero na ordem das posições e copia o resultado para a original com for.

contaOcupadas, **contaRemovidas** e **contaVazias** contam, respectivamente, valores maiores ou iguais a zero, posições REMOVIDO e posições VAZIO.

geraChaves e **geraAusentes** usam 1000000 + sorteio.nextInt(9000000); a primeira descarta duplicadas e conta duplicadasDescartadas, a segunda descarta chaves já existentes.

verificaTeste recebe o código do teste e uma condição, imprime `APROVADO` ou `REPROVADO` e devolve 1 ou 0

executaTestes roda os 26 testes;

executaExperimentos cria new Random(1990) e roda os 5 experimentos;

main chama os dois na ordem exigida.

DIFICULDADES ENCONTRADAS e decisões tomadas:
A maior dificuldade foi entender que a remoção ingênua quebra a busca ao criar um VAZIO no meio da cadeia. O marcador REMOVIDO resolve isso e pode ser reaproveitado, mas insere precisa percorrer o caminho completo antes de gravar para não criar duplicata. Também foi necessário limitar busca e insere a tamanho passos para evitar laço infinito em tabela cheia. Os contadores são zerados apenas por quem chama os métodos, nunca dentro deles, conforme a seção 4.5.

RESULTADO: *TESTES: 26 de 26 aprovados.*

2. Figura 1 - Tabela de 11 posições em quatro momentos

Estado / Pos.	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
inserir 22, 33, 44, 55	22	33	44	55	vazio	vazio	vazio	vazio	vazio	vazio	vazio
removeIngenua(33)	22	vazio	44	55	vazio	vazio	vazio	vazio	vazio	vazio	vazio
remove(33)	22	removido	44	55	vazio	vazio	vazio	vazio	vazio	vazio	vazio
reorganiza	22	44	55	vazio	vazio	vazio	vazio	vazio	vazio	vazio	vazio

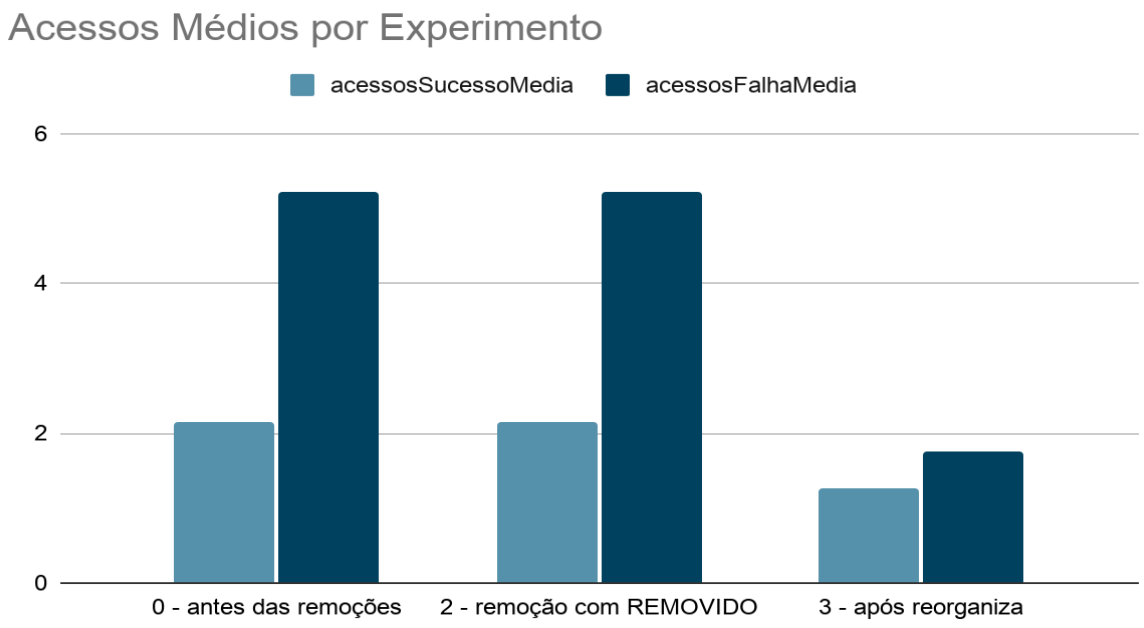
Caminhos de busca da chave 44:

- Em (b): busca(44) examina a posição 0 (22) e depois a posição 1 (VAZIO); para e retorna -1 com 2 acessos. A chave 44 está na posição 2, mas a busca não chega até ela.
- Em (c): busca(44) examina a posição 0 (22), passa pela posição 1 (REMOVIDO) e examina a posição 2 (44); retorna 2 com 3 acessos.
- Em (d), a posição 1 contém 44, a posição 2 contém 55 e a posição 3 fica VAZIO, pois reorganiza reinsere as chaves na ordem das posições em uma tabela limpa.

3. Tabela 1 - Resultados dos experimentos

Experimento	0 — antes das remoções	1 — remoção ingênua	2 — remoção com REMOVIDO	3 — após reorganiza	4 — reinserção
ocupadas	700	431	350	350	700
removidas	0		350	0	151
vazias	300	569	300	650	149
removidasComSucesso		269	350		
falsosNegativos		87	0	0	0
inseridas					350
reaproveitamentos					199
acessosSucessoMedia	2,16		2,16	1,27	2,76
acessosFalhaMedia	5,23		5,23	1,76	

4. Gráfico 1



Experimento(horizontal) / Acessos médios(vertical).

5. Respostas às perguntas do item 7.2

Pergunta 1

Na Figura 1(b), ao executar removeIngenua(33), a posição 1 recebe o valor VAZIO. Quando busca(44) é chamada, ela inicia na posição 44 % 11 = 0 (onde está o 22), avança para a posição 1 e interrompe a procura ao encontrar VAZIO, retornando -1 após 2 acessos. Embora a chave 44 continue armazenada na posição 2, ela se torna inacessível porque a condição de parada dos slides (tabela[posicao] == VAZIO) interpreta qualquer espaço vazio como o fim da cadeia de colisões.

Pergunta 2

No Experimento 1, das 350 tentativas de remoção, apenas 269 tiveram sucesso e ocorreram 87 falsos negativos. As outras 81 chamadas a removeIngenua falharam (retornando false) porque a busca interna interrompia o percurso sempre que esbarrava em um VAZIO deixado por remoções anteriores, sem conseguir alcançar chaves que ainda estavam na tabela. O estado final de 431 posições ocupadas e 569 vazias confirma que somente 269 elementos foram de fato removidos.

Pergunta 3

No Experimento 2, não houve falsos negativos (0) e todas as 350 remoções funcionaram. Isso acontece porque o método remove marca a posição com REMOVIDO em vez de VAZIO. Como visto na Figura 1(c), ao buscar a chave 44, o algoritmo passa pela posição 0 (22), ignora a marcação REMOVIDO na posição 1 e continua até encontrar o 44 na posição 2 (retornando índice 2 com 3 acessos). Dessa forma, a cadeia de sondagem linear não é quebrada e a busca só para em um VAZIO real ou após verificar toda a tabela.

Pergunta 4

Os dados da Tabela 1 mostram que as médias de acesso do Experimento 2 (2,16 no sucesso e 5,23 na falha) permanecem idênticas às do Experimento 0. Como as 350 chaves removidas foram apenas marcadas como REMOVIDO, as 350 chaves restantes continuam nos mesmos índices e exigem exatamente o mesmo percurso para serem encontradas. Da mesma forma, nas buscas sem sucesso, o algoritmo trata REMOVIDO como posição ocupada e percorre o mesmo trajeto até encontrar o VAZIO original.

Pergunta 5

O Experimento 3 mostra que o método reorganiza elimina os marcadores REMOVIDO (0 removidas, 350 ocupadas e 650 vazias) e melhora o desempenho das buscas, reduzindo a média de acessos para 1,27 (sucesso) e 1,76 (falha). Essa melhora ocorre porque os elementos ativos são reinseridos em uma tabela limpa na ordem em que aparecem no vetor, ficando mais próximos de suas posições originais (chave % tamanho). Durante o processo, o método percorre as 1000 posições e aciona insere 350 vezes — uma para cada chave válida (valor >= 0).

Pergunta 6

No Experimento 4, todas as 350 novas chaves foram inseridas, mas houve apenas 199 reaproveitamentos de espaços marcados como REMOVIDO; as outras 151 chaves ocuparam posições VAZIO. O método insere guarda o índice do primeiro REMOVIDO que encontra no caminho, mas se o trajeto de sondagem alcançar um VAZIO antes de passar por alguma posição removida, a chave é gravada diretamente nesse espaço vazio. Assim, das 350 posições removidas inicialmente, 199 foram reutilizadas e 151 permaneceram como REMOVIDO ao final (700 ocupadas, 151 removidas e 149 vazias).

Pergunta 7

O método insere não pode gravar imediatamente no primeiro REMOVIDO encontrado; ele precisa continuar percorrendo a cadeia até achar um VAZIO (ou dar a volta completa) para garantir que a chave ainda não exista na tabela. Na Figura 1(c), por exemplo, a posição 1 está REMOVIDO e a chave 44 está na posição 2. Se insere(tabela, 44) gravasse logo na posição 1, criaria uma duplicata. Ao continuar a varredura, o algoritmo encontra o 44 na posição 2 e retorna -1 corretamente.

Pergunta 8

No material dos slides, a condição original while(tabela[i] != chave && tabela[i] != VAZIO) só conclui a busca ao encontrar a chave ou uma posição VAZIO.Então, se não houver nenhuma posição VAZIO e a chave não existir, o laço não termina. Com o marcador REMOVIDO, isso ocorre tanto em uma tabela totalmente ocupada (teste F1, tamanho 3, {3, 6, 9}) quanto em uma com apenas chaves e posições REMOVIDO (teste F3, {3, REMOVIDO, 9}). Como r(i) = (i + 1) % tamanho (rehash) percorre a tabela de forma circular e REMOVIDO não para a busca, busca(12) passaria repetidamente pelas posições 0, 1 e 2. contador passos resolve isso ao limitar a busca a tamanho posições, ou seja, uma volta completa, retornando -1 após 3 acessos nos testes F1 e F3, ou após 1000 acessos na Tabela 1.

Pergunta 9

As constantes VAZIO (-1) e REMOVIDO (-2) têm funções opostas na sondagem: VAZIO encerra a busca, enquanto REMOVIDO indica um espaço livre para futura inserção que não interrompe o percurso. Se tivessem o mesmo valor, a remoção voltaria a quebrar as buscas como na abordagem ingênua. Além disso, ambos precisam ser valores negativos para não serem confundidos com chaves válidas (que são números positivos de 7 dígitos), como validado na Figura 1 e no teste C1.

Pergunta 10

Um bom critério é chamar o método reorganiza quando a variável removidas atingir 25% do tamanho da tabela (250 posições em uma de 1000) ou quando vazias ficar menor que 20%. Na Tabela 1, o Experimento 2 mostra que juntar 350 posições REMOVIDO (35% da tabela) mantém as médias em 2,16 (sucesso) e 5,23 (falha), o mesmo custo de 700 chaves. Já no

Experimento 3, após reorganiza, removidas volta a zero, vazias sobe para 650 e as médias caem para 1,27 e 1,76. No encadeamento dos slides, cada posição aponta para uma lista encadeada ordenada com as chaves que colidiram naquele índice, e a remoção apenas retira o nó, ajustando o .próximo. Assim, uma posição nunca ocupa o espaço de outra, o que elimina o marcador REMOVIDO e a necessidade de reorganiza.

6. Conclusão

Concluímos que simplesmente gravar VAZIO na posição durante remoções em tabelas hash com rehashing é inadequado, pois separa a sequência de colisão e faz com que a busca termine mais cedo, produzindo falsos negativos (87 no Experimento 1). O uso do REMOVIDO elimina essa falha (0 no Experimento 2) e permite o reaproveitamento de posições em inserções seguintes (199 no Experimento 4), desde que o procedimento de inserção varra a cadeia até sua extremidade antes de gravar para evitar a inclusão de chaves duplicadas. Contudo, como as pesquisas continuam a atravessar as posições marcadas REMOVIDO, o custo médio permanece alto. Portanto, recomendamos chamar o método reorganiza quando o total de entradas removidas atingir aproximadamente 25% do tamanho da tabela ou quando a proporção de posições vazias for menor que 20%. Essa reorganização reduz os acessos médios, conforme evidenciado no Experimento 3.

7. Declaração

Declaramos que o código e o texto deste trabalho foram produzidos pelos integrantes do grupo, sem uso de ferramenta de inteligência artificial generativa e sem cópia de outros grupos ou de fontes externas.

- Mariana Schneider Sobrinho - 40107348
- Heitor Roberto Gonçalves - 40113496
- Josue A. Nonalaya - 4019965

8. Referências

Slides de Tabela Hash da disciplina.
Especificação do Trabalho 3 - Remoção em Tabela Hash com Rehashing.